Projet Mastermind

Gaspard Duc-Jacquet
Mattéo Fierri

Groupe B

Année 2024-2025

Table des matières

1 codebreaker0 **3**
- 1.1 Espérance, loi analytique . . . 3
- 1.2 Test,validation de la loi analytique . . . 3

2 codebreaker1 **5**
- 2.1 Espérance, loi analytique . . . 5
- 2.2 Test,validation de la loi analytique . . . 5
- 2.3 Comparaison codebreaker1,codebreaker0, visualisation du gain 8

3 codebreaker2 **9**
- 3.1 Fonctionnement du codebreaker2 . . . 9
- 3.2 Mesure des performances, comparaison à codebreaker1 . . . 9

4 codemaker2 **12**
- 4.1 Fonctionnement du codemaker2 . . . 12
- 4.2 Mesure des performances, comparaison à codemaker1 . . . 13

5 codebreaker3 **17**
- 5.1 Exemple de test optimal . . . 17
- 5.2 Fonctionnement du codebreaker3 . . . 17
- 5.3 Mesure des performances, comparaison à codebreaker2 . . . 18
  - 5.3.1 Face au codemaker2 . . . 18
  - 5.3.2 Face au codemaker1 . . . 21

6 Interface graphique **24**
- 6.1 Menu . . . 25
- 6.2 Interface du jeu . . . 27
  - 6.2.1 Bouton de couleurs . . . 27
  - 6.2.2 Tableau de jeu . . . 27
  - 6.2.3 Validation . . . 28
- 6.3 Fenêtre de Résultat . . . 29

1 `codebreaker0`

1.1 Espérance, loi analytique

Le nombre total de combinaisons possibles est donné par :

$$N = \texttt{len(common.COLORS)}^{\texttt{common.LENGHT}} \tag{1}$$

Pour cette version du codebreaker qui choisit aléatoirement une combinaison, chaque tentative est indépendante et a une probabilité $\frac{1}{N}$ de réussir. Ainsi en notant X_0 la variable aléatoire comptant le nombre d'essais nécessaire du `codebreaker0` pour trouver la bonne combinaison, X_0 suit une loi géométrique de paramètre $p = \frac{1}{N}$, ainsi l'espérance de X_0 est donné par :

$$E(X_0) = \frac{1}{p} = \texttt{len(common.COLORS)}^{\texttt{common.LENGHT}} \tag{2}$$

1.2 Test,validation de la loi analytique

On peut valider la loi (1) de manière empirique en simulant un grand nombre d'expériences pour différentes valeurs de `len(common.COLORS)` et `common.LENGHT`.

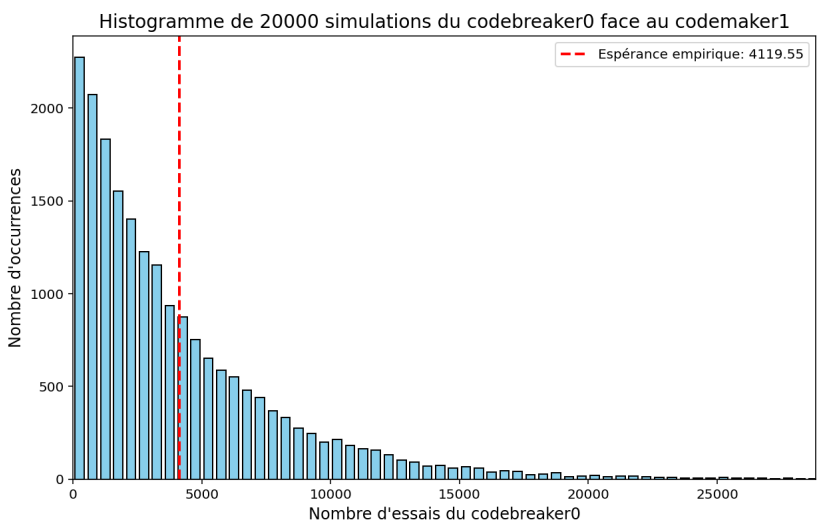

FIGURE 1 – Histogramme du nombre d'essais avec `common.LENGHT=4` et `len(common.COLORS) = 8` (paramètres de base).

On a bien $E(X_0) \simeq 8^4 = 4096$

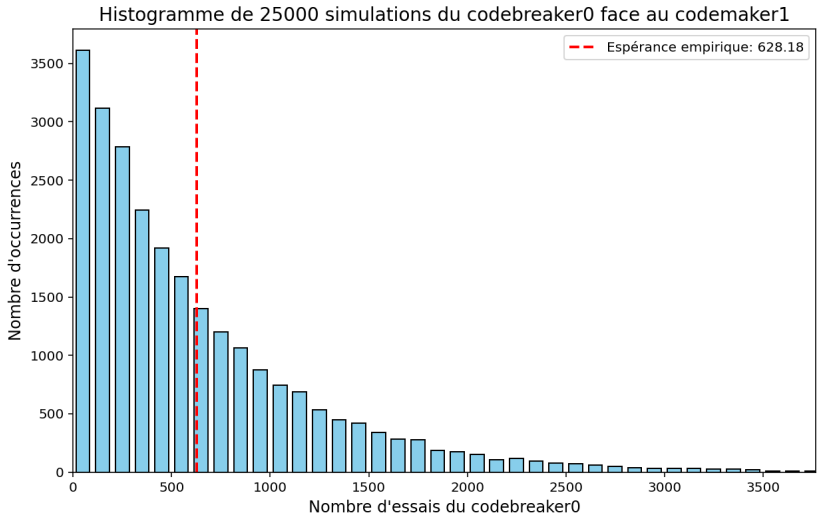

FIGURE 2 – Histogramme du nombre d'essais avec `common.LENGHT = 4` et `len(common.COLORS) = 5`.

On a bien $E(X_0) \simeq 5^4 = 625$

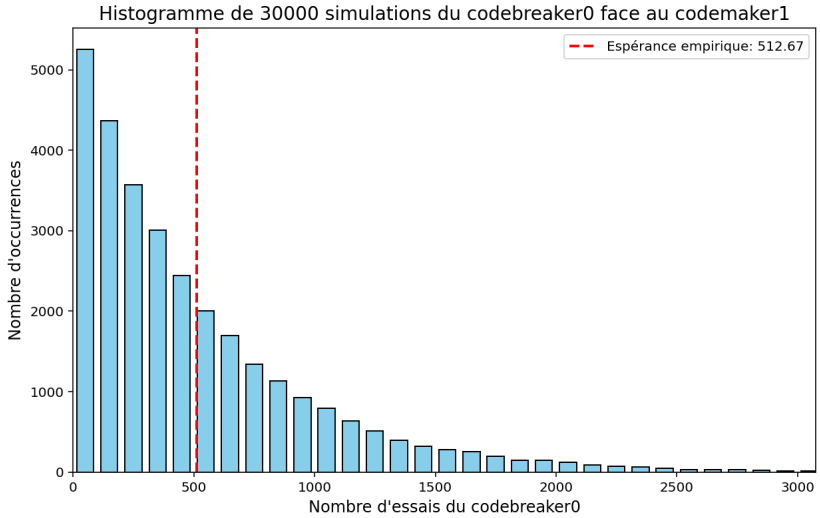

FIGURE 3 – Histogramme du nombre d'essais avec `common.LENGHT = 3` et `len(common.COLORS) = 8`.

On a bien $E(X_0) \simeq 8^3 = 512$

On constate ainsi que nos résultats empiriques suivent bien la loi de probabilité (2).
Cette observation est du à la loi des grands nombres qui énonce que pour un très grand nombre d'expériences, l'espérance empirique converge vers l'espérance théorique.

2 `codebreaker1`

2.1 Espérance, loi analytique

Le nombre total de combinaisons possibles N est toujour donné par la formule (1)

Dans cette version, le codebreaker ne teste jamais deux fois la même combinaison. À chaque essai, il élimine une possibilité. Puisque chaque combinaison est choisie de manière uniforme parmi celles restantes, la position de la combinaison correcte dans la liste des essais suit une distribution uniforme sur {1,2,...,N}.Ainsi en notant X_1 la variable aléatoire comptant le nombre d'essais nécessaire au `codebreaker1` pour trouver la bonne combinaison, X_1 suit une loi uniforme de paramètre $n = N$, ainsi l'espérance de X_1 est donné par :

$$E(X_1) = \frac{N+1}{2} = \frac{\texttt{len(common.COLORS)}^{\texttt{common.LENGHT}} + 1}{2} \tag{3}$$

Ainsi on s'attend à ce que le `codebreaker1` mette en moyenne deux fois moins d'essais que le `codebreaker0` pour trouver la bonne solution.

2.2 Test,validation de la loi analytique

On peut valider la loi (3) de manière empirique en simulant un grand nombre d'expériences pour différentes valeurs de `len(common.COLORS)` et `common.LENGHT`.

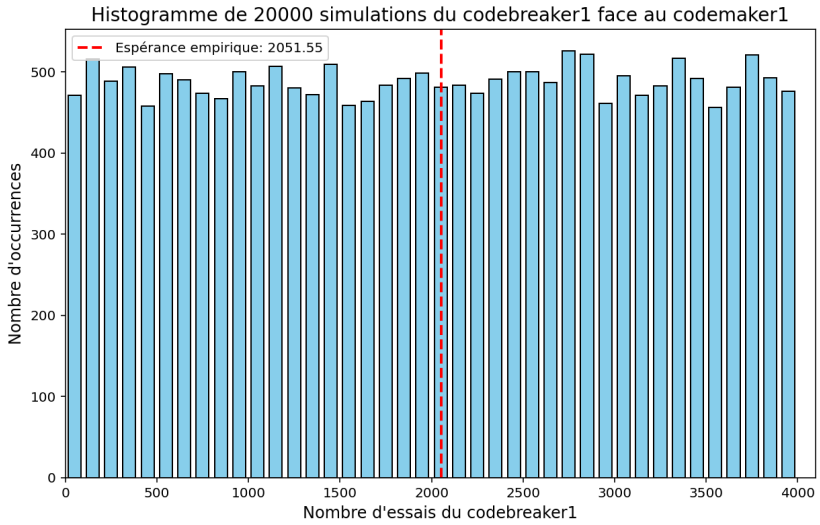

FIGURE 4 – Histogramme du nombre d'essais avec `common.LENGHT=4` et `len(common.COLORS) = 8` (paramètres de base).

$$\text{On a bien } E(X_1) \simeq \frac{8^4+1}{2} = 2048.5$$

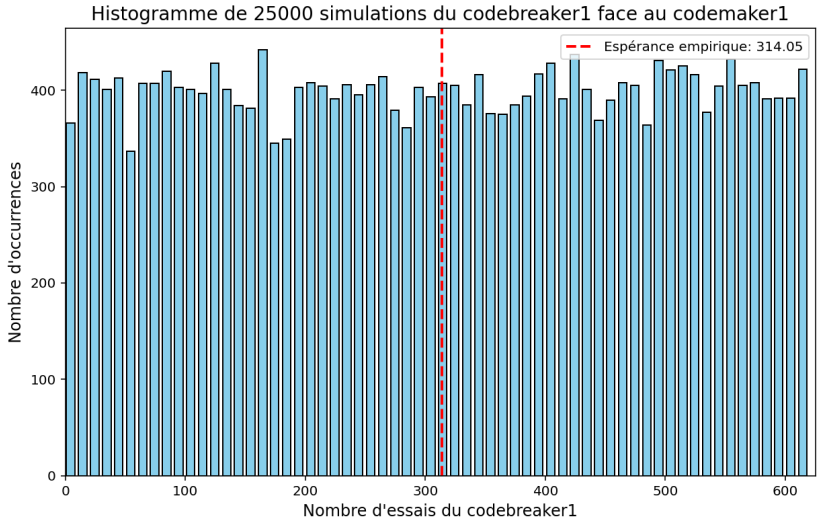

FIGURE 5 – Histogramme du nombre d'essais avec `common.LENGHT=4` et `len(common.COLORS) = 5`

$$\text{On a bien } E(X_1) \simeq \frac{5^4+1}{2} = 313$$

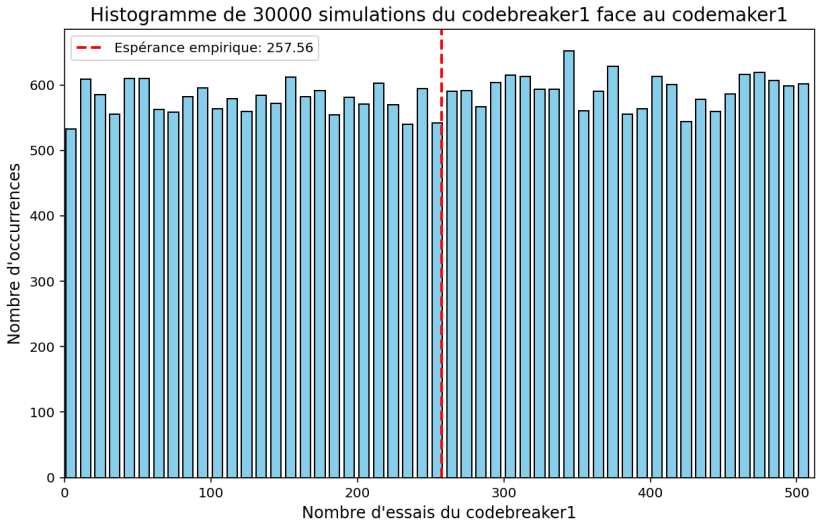

FIGURE 6 – Histogramme du nombre d'essais avec `common.LENGHT=3` et `len(common.COLORS) = 8`

On a bien $E(X_1) \simeq \frac{8^3+1}{2} = 256.5$

2.3 Comparaison `codebreaker1`,`codebreaker0`, visualisation du gain

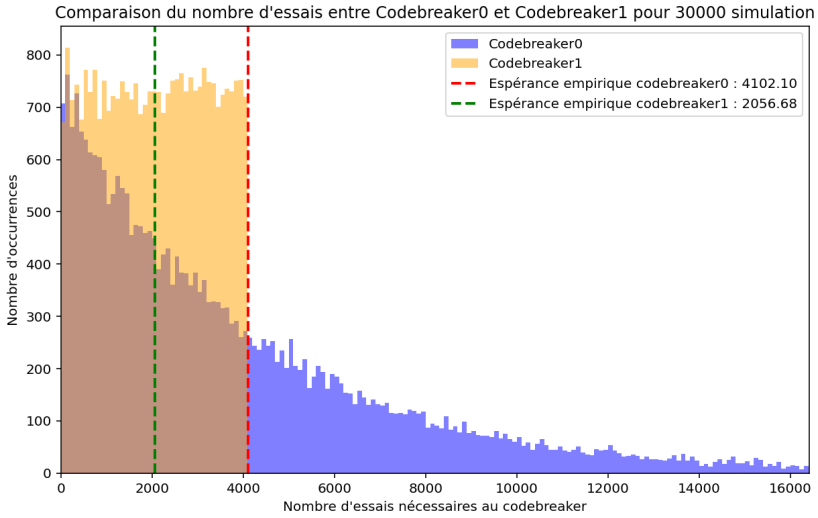

FIGURE 7 – Histogramme du nombre d'essais pour `codebreaker1` et `codebreaker0` avec `common.LENGHT=4` et `len(common.COLORS) = 8` (paramètres de base)

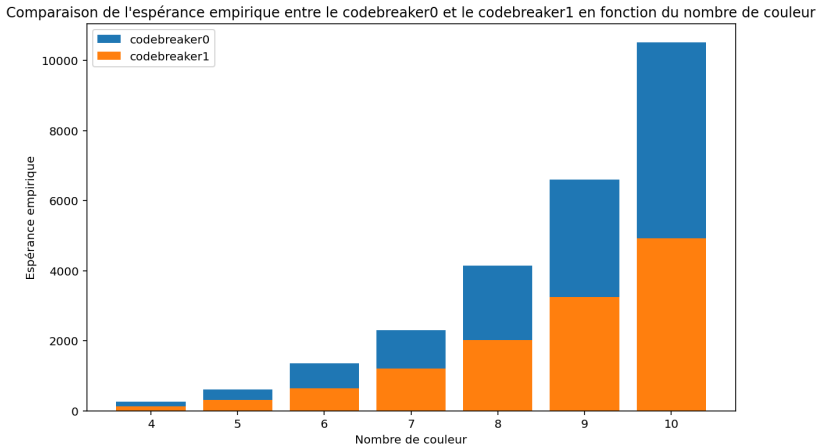

FIGURE 8 – Histogramme de l'espérance empirique estimé à partir de 1000 simulation (pour chaque liste de couleur) pour `codebreaker1` et `codebreaker0` avec `common.LENGHT=4` et `len(common.COLORS)`$\in [\![4, 10]\!]$

Grâce aux figures 7 et 8, on constate que, comme prédit, le `codebreaker1` a en moyenne besoin de deux fois moins d'essais que le `codebreaker0` pour trouver la solution, ce qui valide notre théorie.

3 `codebreaker2`

3.1 Fonctionnement du `codebreaker2`

Dans cette version, le `codebreaker` choisit à chaque tour une combinaison aléatoire parmi l'ensemble des combinaisons encore compatibles avec les retours précédents. Pour mettre à jour cet ensemble, on conserve uniquement les combinaisons qui auraient donné exactement la même évaluation que celle obtenue, si elles avaient été comparées à la combinaison testée. Autrement dit, une combinaison reste possible si elle produirait, face à la combinaison testée, le même nombre de couleurs bien placées et le même nombre de couleurs mal placées que l'évaluation réellement observée.

3.2 Mesure des performances, comparaison à `codebreaker1`

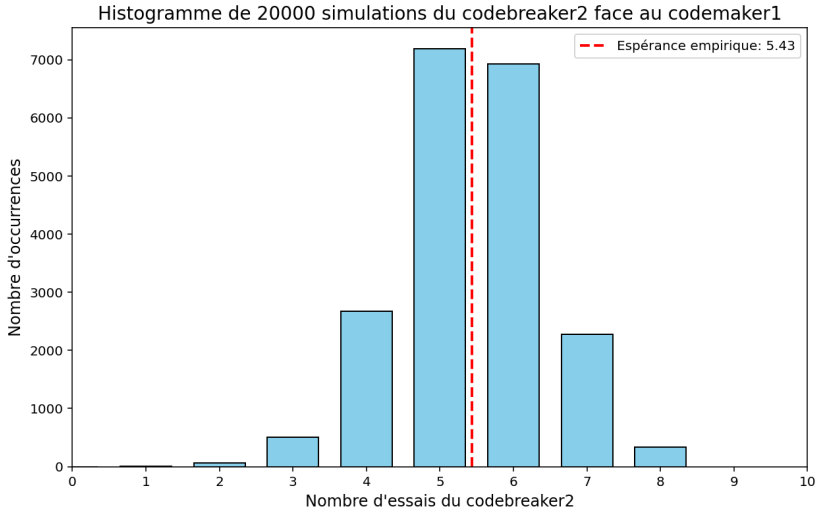

FIGURE 9 – Histogramme du nombre d'essais avec `common.LENGHT=4` et `len(common.COLORS) = 8` (paramètres de base).

On constate que l'espérance empirique avec les paramètres de base est de 5.43. En notant X_2 la variable aléatoire comptant le nombre d'essais nécessaires au `codebreaker2` pour trouver la bonne combinaison, on peut estimer le gain entre le `codebreaker1` et le `codebreaker2` par :

$$\text{Gain} = \frac{E[X_1] - E[X_2]}{E[X_1]} \times 100 = \frac{2048.5 - 5.43}{2048.5} \times 100 \simeq 99.7\%$$

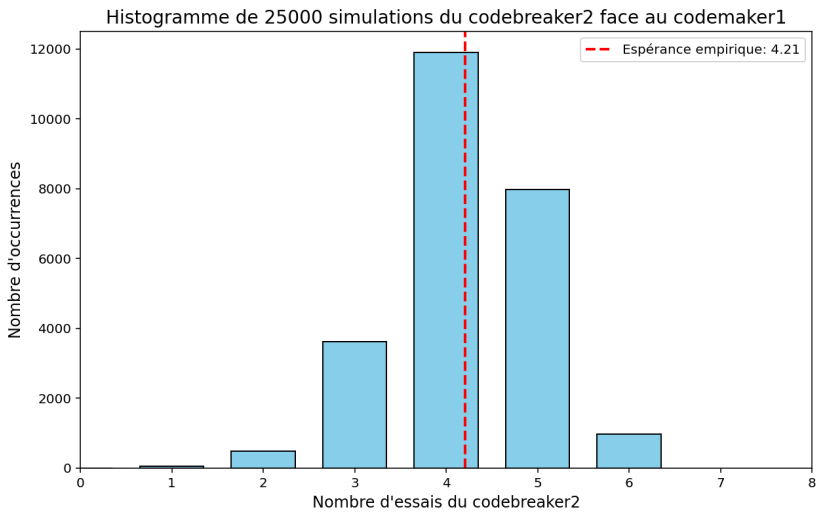

FIGURE 10 – Histogramme du nombre d'essais avec `common.LENGHT=4` et `len(common.COLORS) = 5`

On constate que l'espérance empirique avec ces paramètres est de 4.21. On peut de la même manière estimer le gain entre le `codebreaker1` et le `codebreaker2` par :

$$\text{Gain} = \frac{E[X_1] - E[X_2]}{E[X_1]} \times 100 = \frac{313 - 4.21}{313} \times 100 \simeq 98.6\%$$

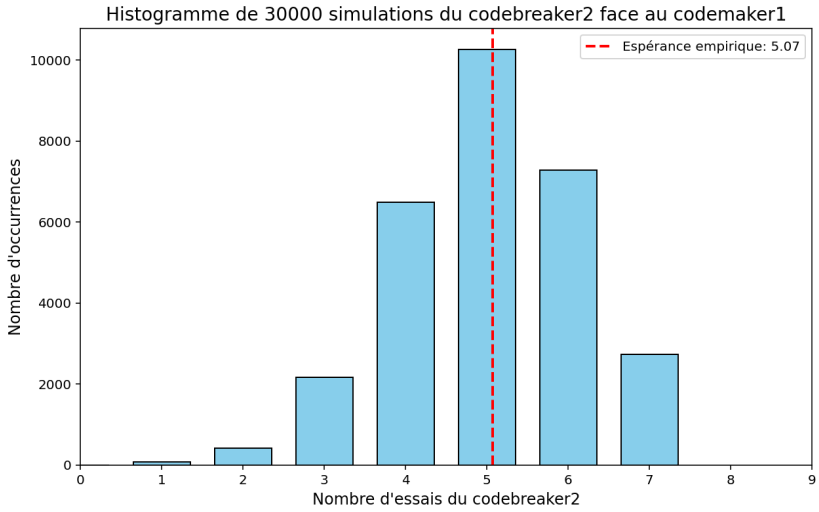

FIGURE 11 – Histogramme du nombre d'essais avec `common.LENGHT=3` et `len(common.COLORS)=8`

On constate que l'espérance empirique avec ces paramètres est de 5.07. De la même manière, on peut estimer le gain entre le `codebreaker1` et le `codebreaker2` par :

$$\text{Gain} = \frac{E[X_1] - E[X_2]}{E[X_1]} \times 100 = \frac{256.5 - 5.07}{256.5} \times 100 \simeq 98.0\%$$

On constate ainsi que l'on obtient un gain significatif entre le `codebreaker1` et le `codebreaker2` et donc que le `codebreaker2` est beaucoup plus efficace que le `codebreaker1`.

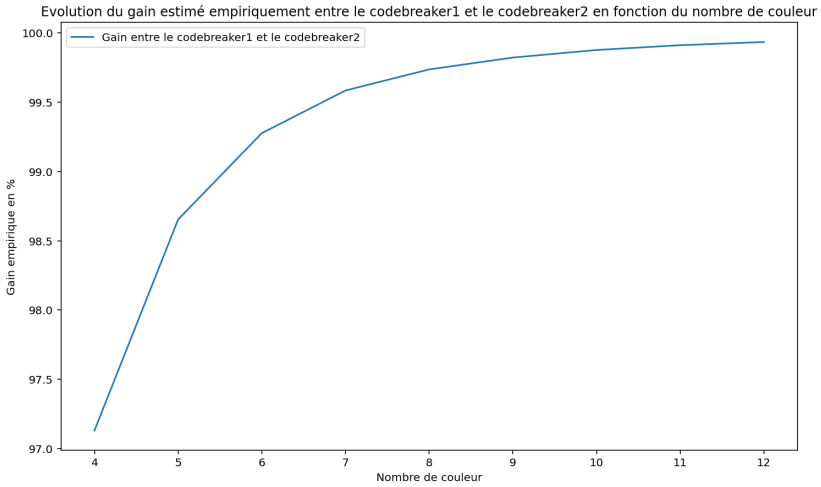

FIGURE 12 – Evolution du gain estimé empiriquement à partir de 5000 simulation entre le `codebreaker1` et le `codebreaker2` en fonction du nombre de couleur avec `common.LENGHT=4` et `len(common.COLORS)`$\in [\![4, 12]\!]$

On constate également avec la figure 12 que plus le nombre de possibilités est grand (on augmente le nombre de couleurs), plus le gain entre le `codebreaker1` et le `codebreaker2` est significatif, cela s'explique par le fait que plus le nombre de possibilités est grand, plus la probabilité de trouver la solution en choisissant aléatoirement est faible et ainsi le `codebreaker2` est d'autant plus efficace comparé au `codebreaker1`.

4 `codemaker2`

4.1 Fonctionnement du `codemaker2`

Dans cette version, on autorise le `codemaker2` à changer de combinaison en cours de partie, à condition de respecter les évaluations déjà données auparavant.

Pour prolonger la partie le plus longtemps possible, nous avons adopté la stratégie suivante :

Mémorisation des combinaisons possibles

Tout au long du jeu, le `codemaker2` conserve en mémoire l'ensemble des combinaisons encore possibles, c'est-à-dire celles qui restent compatibles avec les évaluations précédentes.

Analyse de chaque nouvelle proposition

À chaque tentative du `codebreaker`, le `codemaker2` évalue, pour chaque combinaison encore possible, le nombre de solutions restantes après l'évaluation du coup joué.

Sélection de la combinaison optimale

Après avoir effectué tous ces calculs, le `codemaker2` choisit la combinaison qui maintient le plus grand nombre de solutions possibles.
Cette stratégie ralentit ainsi l'élimination des combinaisons et prolonge la durée de la partie.

Vous retrouverez également un fichier de test sous le nom `test` qui nous permet de vérifier que le `codemaker2` ne triche pas de manière évidente.

4.2 Mesure des performances, comparaison à `codemaker1`

Pour mesurer et comparer les performances de nos `codemaker` nous allons les confronter au `codebreaker2` qui est pour l'instant le seul `codebreaker` qui ne joue pas aléatoirement.

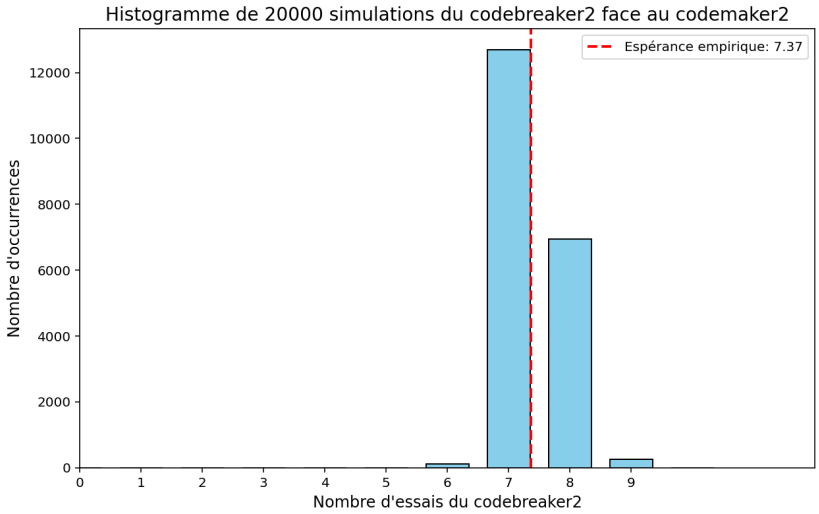

FIGURE 13 – Histogramme du nombre d'essais avec `common.LENGHT=4` et `len(common.COLORS) = 8` (paramètres de base).

On constate que l'espérance empirique avec les paramètres de base est de 7.37. En notant $X_{2,2}$ la variable aléatoire comptant le nombre d'essais nécessaires au `codebreaker2` face au `codemaker2` pour trouver la bonne combinaison et $X_{2,1}$ la variable aléatoire comptant le nombre d'essais nécessaires au `codebreaker2`

face au `codemaker1` pour trouver la bonne combinaison, on peut estimer le gain en termes de longueur de partie entre le `codemaker1` et le `codemaker2` par :

$$\text{Gain} = \frac{E[X_{2,2}] - E[X_{2,1}]}{E[X_{2,1}]} \times 100 = \frac{7.37 - 5.43}{5.43} \times 100 \simeq 35.7\%$$

Ou autrement dit le `codebreaker2` met en moyenne 1.94 essai de plus face au `codemaker2` que face au `codemaker1` avec ces paramètres.

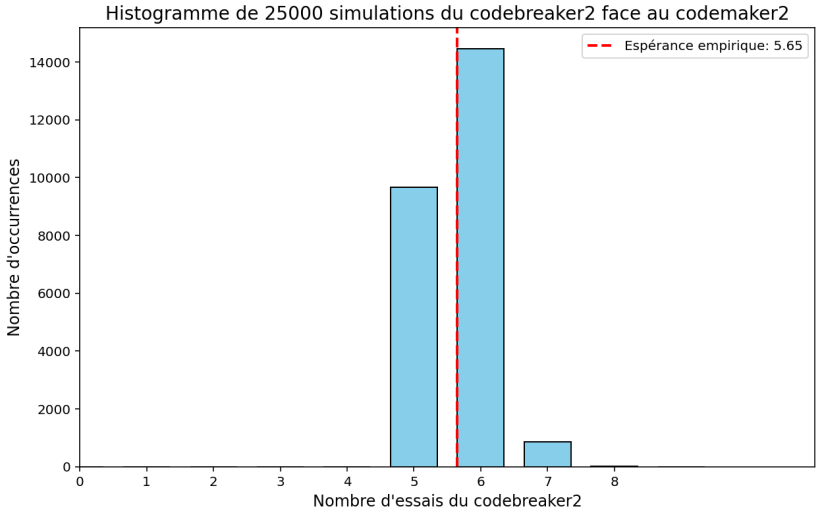

FIGURE 14 – Histogramme du nombre d'essais avec `common.LENGHT=4` et `len(common.COLORS) = 5`

On constate que l'espérance empirique avec ces paramètres est de 5.65. De la même manière on peut estimer le gain entre le `codemaker1` et le `codemaker2` par :

$$\text{Gain} = \frac{E[X_{2,2}] - E[X_{2,1}]}{E[X_{2,1}]} \times 100 = \frac{5.65 - 4.21}{4.21} \times 100 \simeq 34.7\%$$

Ou autrement dit le `codebreaker2` met en moyenne 1.44 essai de plus face au `codemaker2` que face au `codemaker1` avec ces paramètres.

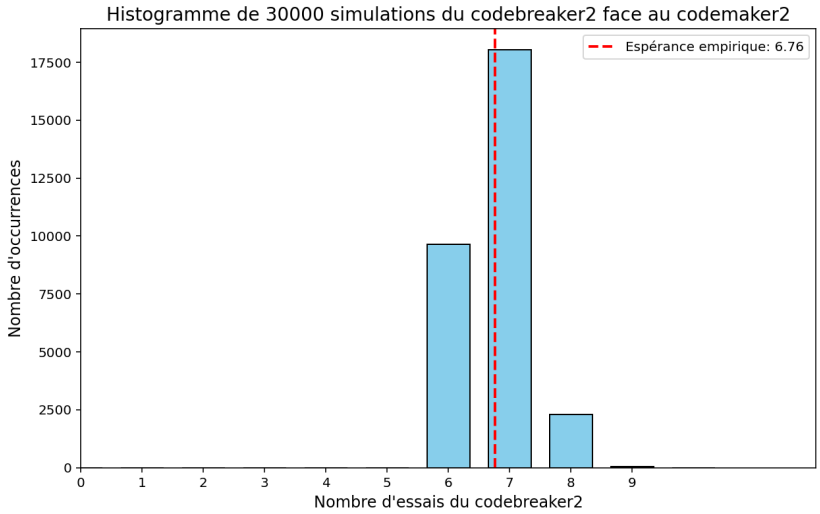

FIGURE 15 – Histogramme du nombre d'essais avec `common.LENGHT=3` et `len(common.COLORS) = 8`

On constate que l'espérance empirique avec ces paramètres est de 6.34. De la même manière on peut estimer le gain entre le `codemaker1` et le `codemaker2` par :

$$\text{Gain} = \frac{E[X_{2,2}] - E[X_{2,1}]}{E[X_{2,1}]} \times 100 = \frac{6.34 - 5.07}{5.07} \times 100 \simeq 31.5\%$$

Ou autrement dit le `codebreaker2` met en moyenne 1.60 essai de plus face au `codemaker2` que face au `codemaker1` avec ces paramètres.

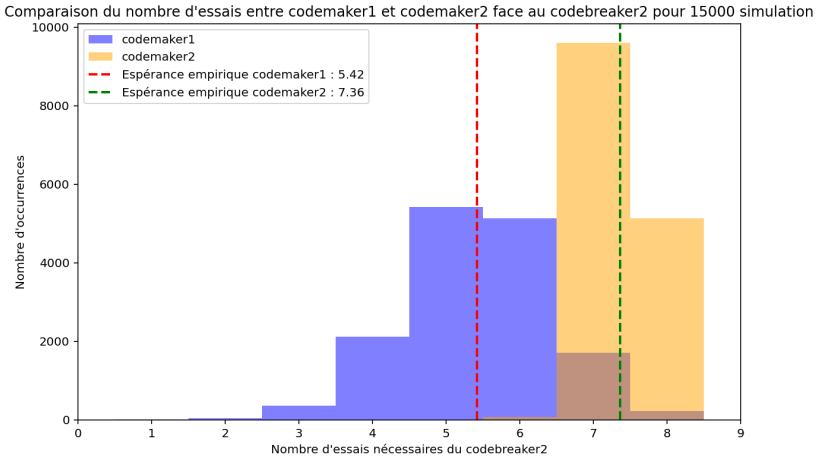

FIGURE 16 – Histogramme du nombre d'essais du `codebreaker2` face au `codemaker1` et au `codemaker2` `common.LENGHT=4` et `len(common.COLORS) = 8`(paramètres de base)

On constate avec cette figure que le `codemaker2` remplit bien son rôle, en effet on voit qu'avec le `codemaker2`, le `codebreaker2` n'arrive jamais à finir une partie en moins de 6 essais et que très rarement à finir une partie en moins de 7 essais, contrairement au `codemaker1`

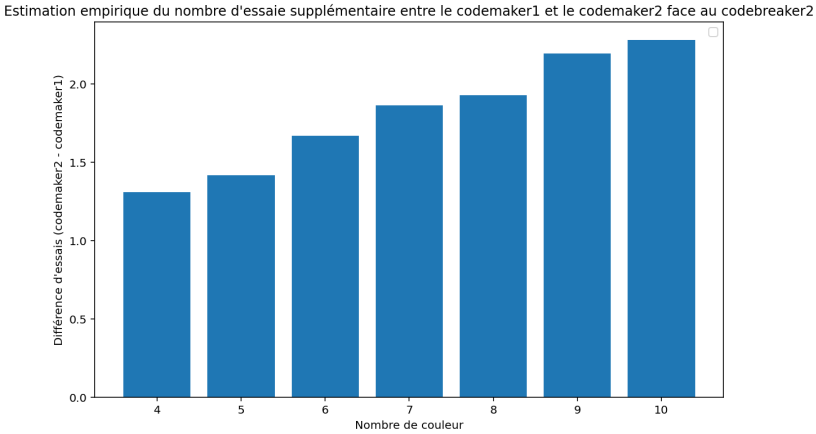

FIGURE 17 – Evolution du nombre d'essai supplémentaire entre le `codemaker1` et le `codemaker2` face au `codebreaker2` calculé empiriquement à partir de 1000 simulation en fonction du nombre de couleur avec `common.LENGHT=4` et `len(common.COLORS)`$\in [\![4, 10]\!]$

On constate ainsi que plus le nombre de possibilités est grand (on augmente le

nombre de couleurs), plus le `codemaker2` est efficace et arrive à prolonger les parties par rapport au `codemaker1`.

5 `codebreaker3`

Il n'est pas toujours optimal pour le `codebreaker` de ne tester que des combinaisons encore possibles d'après les évaluations précédentes. Parfois, essayer une combinaison que l'on sait déjà incorrecte peut permettre d'obtenir plus d'informations que n'importe quelle combinaison valide restante.

5.1 Exemple de test optimal

Prenons l'ensemble de couleurs `COLORS = ['R', 'V', 'B', 'J', 'N']` et supposons que la solution soit `'NNNN'`.

Première proposition :

Le `codebreaker` propose `'NNRR'`. L'évaluation obtenue est (2,0), ce qui signifie que deux 'N' sont bien placés et qu'aucune autre lettre n'est correcte.

Deuxième proposition :

Une approche classique consisterait à ne tester que des combinaisons qui respectent l'évaluation précédente. Cependant, supposons que le `codebreaker` propose `'RVBJ'`, qui est une combinaison que l'on sait déjà incorrecte par rapport à `'NNRR'`. L'évaluation obtenue est (0,0), indiquant que ni `'R'`, ni `'V'`, ni `'B'`, ni `'J'` ne font partie de la solution.

Grâce à cette proposition, le `codebreaker` peut immédiatement conclure que la seule couleur présente dans la solution est `'N'`. Il peut donc déduire directement que la réponse correcte est `'NNNN'`, en seulement deux essais.

Dans ce cas, tester une combinaison apparemment inutile a permis de découvrir toute la solution d'un seul coup, alors qu'une approche classique aurait pu nécessiter plus d'essais.

5.2 Fonctionnement du `codebreaker3`

Dans cette version, le `codebreaker3` test à chaque tour la combinaison dont l'évaluation lui rapportera le plus d'informations dans le pire des cas. C'est-à-dire la combinaison qui lui permettra de réduire au maximum le nombre de combinaisons possibles après l'évaluation.On autorise notamment le `codebreaker3` à tester des combinaisons impossibles au vu des évaluations précédentes.

Pour déterminer à chaque tour le test optimal, nous avons adopté la stratégie suivante :

Mémorisation des combinaisons possibles

Tout au long du jeu, le `codebreaker3` conserve en mémoire l'ensemble des combinaisons encore possibles, c'est-à-dire celles qui restent compatibles avec les évaluations précédentes.

Analyse de chaque combinaison

À chaque tour, le `codebreaker3` évalue pour chaque combinaison existante l'ensemble des réponses qu'elle pourrait générer en fonction des solutions encore envisageables. Il calcule ensuite combien de combinaisons resteraient possibles après chaque évaluation.

Selection de la combinaison optimale

Après avoir effectué ces calculs, le `codebreaker3` choisit la combinaison qui minimise la taille du plus grand ensemble de solutions restantes. Autrement dit, il sélectionne celle qui garantit l'élimination du plus grand nombre de possibilités, quel que soit le retour du codemaker.

5.3 Mesure des performances, comparaison à `codebreaker2`

5.3.1 Face au `codemaker2`

Dans cette partie on s'intéresse aux performances du `codebreaker3` face au `codemaker2`, cependant les parties opposant ces deux versions sont très longues car les deux programmes sont gourmands en calcul, nous avons donc réduit le nombre de simulations par figure.

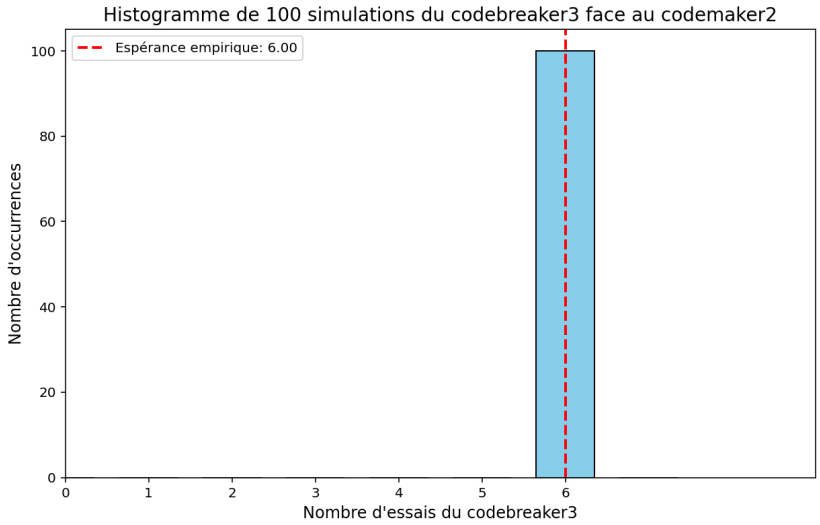

FIGURE 18 – Histogramme du nombre d'essais avec `common.LENGHT=4` et `len(common.COLORS) = 8` (paramètres de base).

On constate qu'à chaque partie le `codebreaker3` met exactement le nombre de coups minimal qu'a pu effectuer le `codebreaker2` avec ces paramètres.

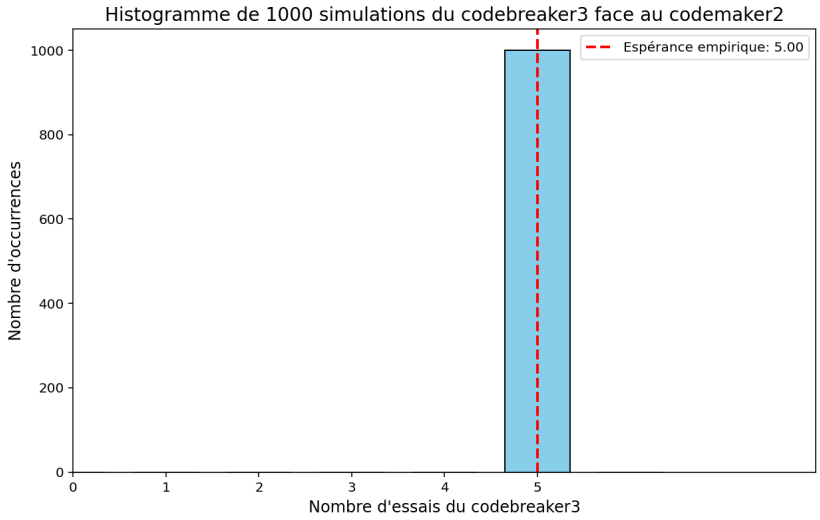

FIGURE 19 – Histogramme du nombre d'essais avec `common.LENGHT=4` et `len(common.COLORS) = 5`

Même constat que pour la figure précédente, à chaque partie le `codebreaker3`

met exactement le nombre de coups minimal qu'a pu effectuer le `codebreaker2` avec ces paramètres.

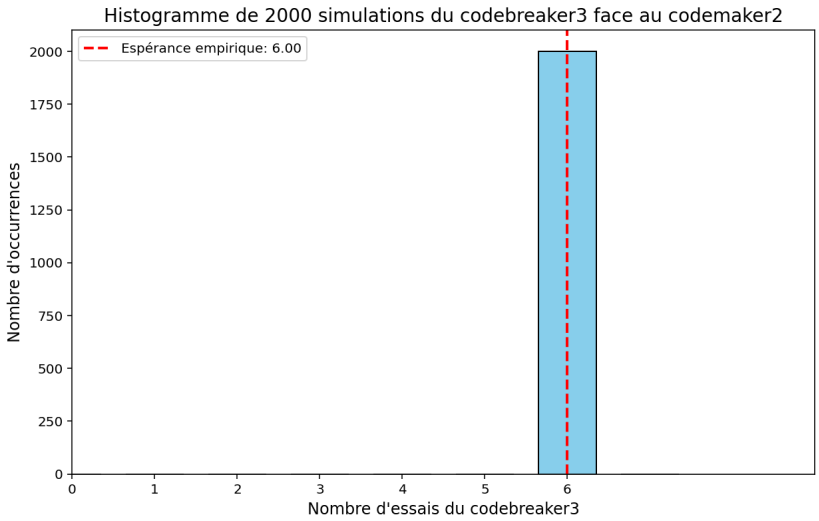

FIGURE 20 – Histogramme du nombre d'essais avec `common.LENGHT=3` et `len(common.COLORS) = 8`

Comme pour les figures précédentes, à chaque partie le `codebreaker3` met exactement le nombre de coups minimal qu'a pu effectuer le `codebreaker2` avec ces paramètres.

On peut également visualiser ce résultat avec la figure suivante :

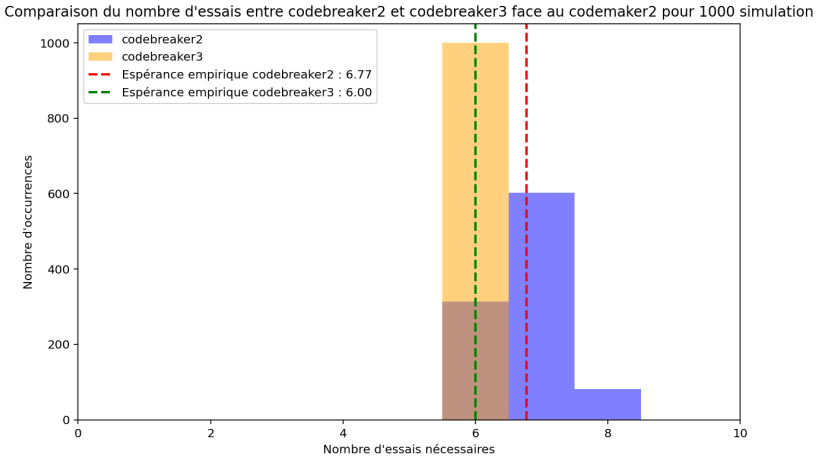

FIGURE 21 – Histogramme du nombre d'essais du `codebreaker2` et du `codebreaker3` face au `codemaker2` avec `common.LENGHT=3` et `len(common.COLORS) = 8`

TABLE 1 – Table des espérances calculées empiriquement face au `codemaker2`

	`C=8` et `L=4`	`C=5` et `L=4`	`C=8` et `L=3`
`codebreaker2`	7,37	5,65	6,76
`codebreaker3`	6,00	5,00	6,00

Ces résultats ne sont pas surprenants, car `codebreaker3` et `codemaker2` jouent de manière optimale, et il n'existe qu'une seule stratégie optimale. Cela confirme donc que ces deux version adoptent bien un comportement optimal.

5.3.2 Face au `codemaker1`

Dans cette partie, on s'intéresse aux performances du `codebreaker3` face au `codemaker1`. Les parties opposant ces deux versions sont elles aussi très longues, car `codebreaker3` est très gourmand en calcul. Nous avons donc également, dans cette section, réduit le nombre de simulations par figure.

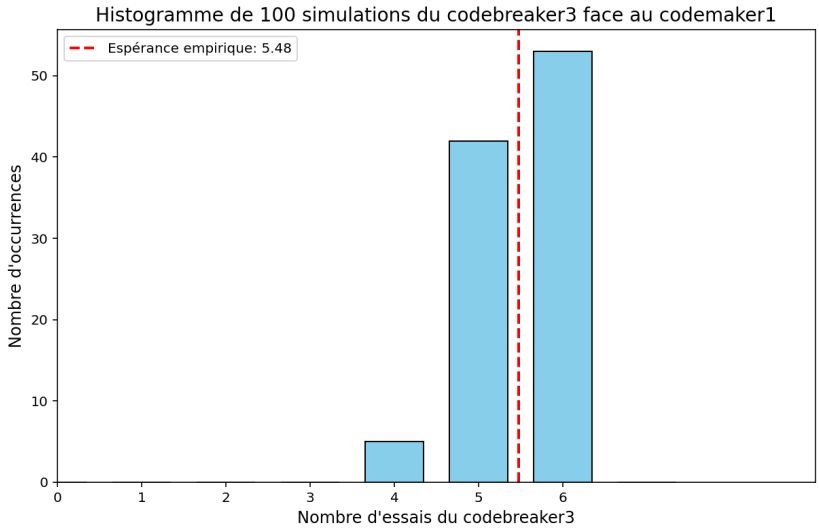

FIGURE 22 – Histogramme du nombre d'essais avec `common.LENGHT=4` et `len(common.COLORS) = 8` (paramètres de base).

On constate que l'espérance du `codebreaker3` est très légèrement supérieure à celle du `codebreaker2` (5,48 contre 5,43)

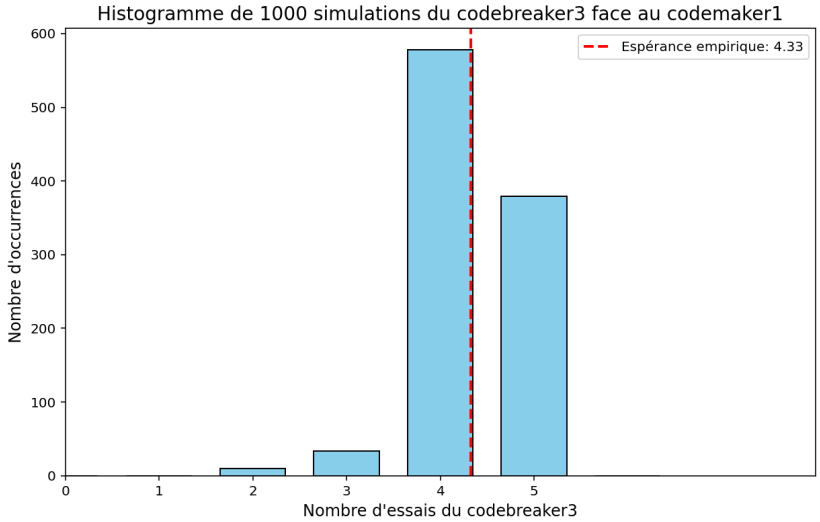

FIGURE 23 – Histogramme du nombre d'essais avec `common.LENGHT=4` et `len(common.COLORS) = 5`

On constate que l'espérance du `codebreaker3` est très légèrement supérieure à

celle du `codebreaker2` (4,33 contre 4,21)

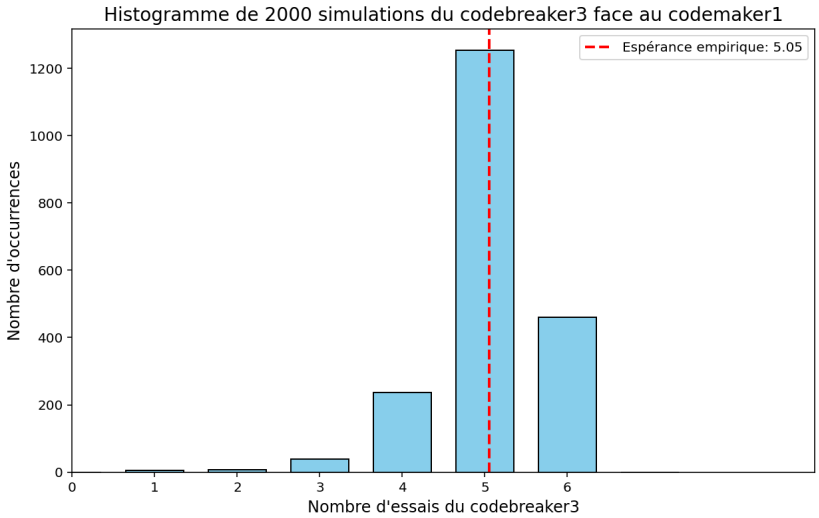

FIGURE 24 – Histogramme du nombre d'essais avec `common.LENGHT=3` et `len(common.COLORS)=8`

On constate que l'espérance du `codebreaker3` et très légèrement inférieure à celle du `codebreaker2` (5,05 contre 5,07).

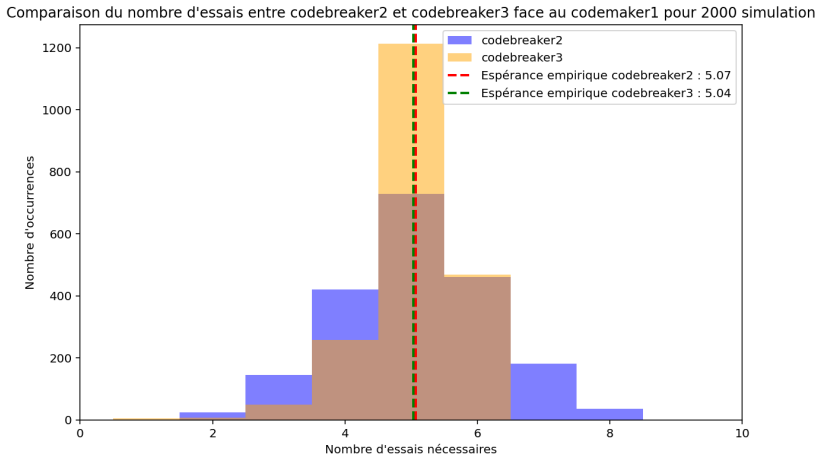

FIGURE 25 – Histogramme du nombre d'essais du `codebreaker2` et du `codebreaker3` face au `codemaker1` avec `common.LENGHT=3` et `len(common.COLORS) = 8`

On constate avec cette figure que le `codebreaker3` à un nombre de coup joué

plus régulier qui varie entre 1 et 6, contrairement au `codebreaker2` qui varie entre 1 et 8.

Table 2 – Table des espérances calulées empiriquement face au `codemaker1`

	`C=8` et `L=4`	`C=5` et `L=4`	`C=8` et `L=3`
`codebreaker2`	5,43	4,21	5,07
`codebreaker3`	5,48	4,33	5,05

Ces résultat nous montrent que le `codebreaker3`, apporte un réel gain d'efficacité par rapport au `codebreaker2` face à un `codemaker` qui triche.Ses performances sont également quasi similaires au `codebreaker2` face à un `codemaker` qui ne triche pas.

6 Interface graphique

Finalement, nous avons implanté une interface graphique pour permettre à un joueur d'affronter un de nos codemakers. Pour ce faire, nous avons utilisé la librairie `Tkinter`.

Pour vous aider à comprendre comment nous avons structuré notre code voici un graphe résumant les appels des fonctions principales :

La figure ci-dessous présente l'enchaînement principal des fonctions de l'interface graphique. Ce diagramme met en évidence la logique de navigation entre les différentes fenêtres (menu, jeu, fin) et les interactions utilisateur qui déclenchent les appels de fonctions *(représenté par des flèches)*

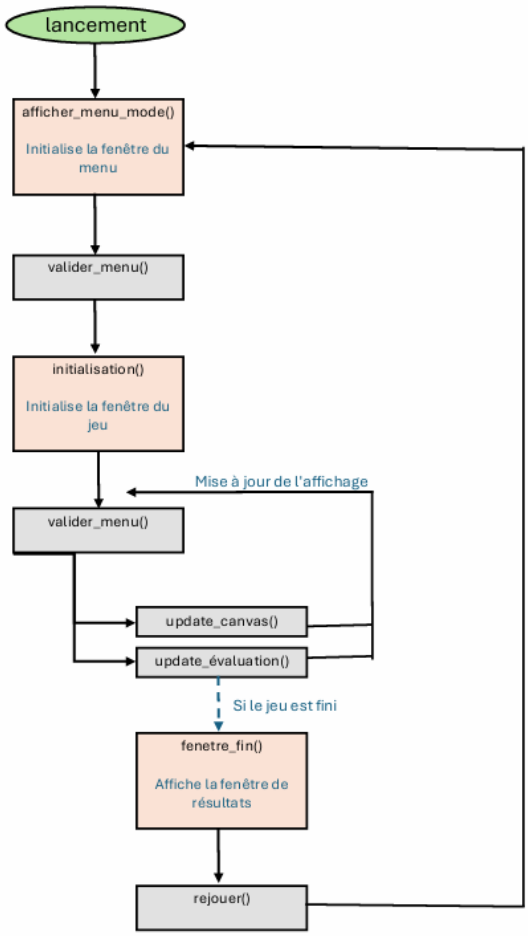

FIGURE 26 – Architecture logique des appels dans l'interface graphique

6.1 Menu

On a choisi de créer un menu avant de lancer le jeu pour choisir :

— **Le mode de jeu :** facile ou difficile en appelant respectivement codemaker 1 ou 2, pour ce faire, on utilise deux boutons qui, lorsqu'ils sont clické change le codemaker appelé.

— **Le nombre d'essais :** Grace à un curseur on choisit le nombre d'essais (`nombre_essais`) pour trouver le résultat et on passe cette variable en global pour la réutiliser par la suite.

— **Le nombre de couleurs :** De façon analogue, on choisit le nombre de couleurs (`nombre_couleurs`) avec un curseurs.

Finalement, on utilise le bouton `valider` afin d'initialiser le codemaker choisi et de modifier la liste `COLORS` de common.py avec notre nouvelle liste de couleurs de longueur `nombre_couleurs`. On ferme la fenêtre du menu et on ouvre la fenêtre du jeu.

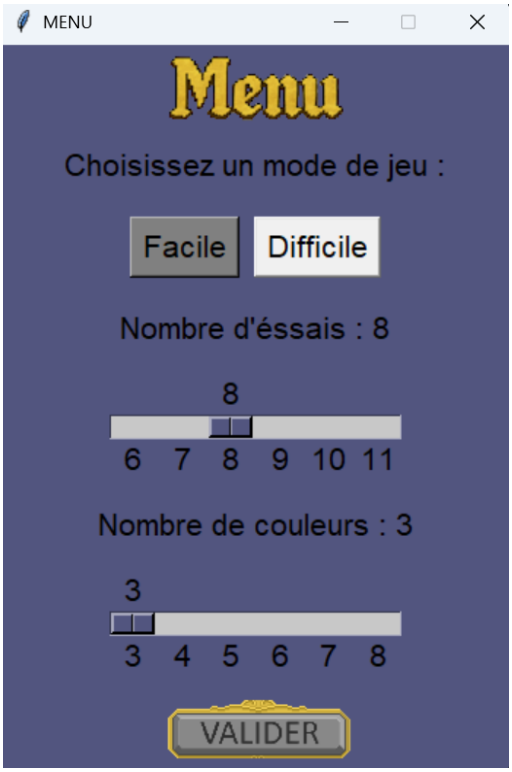

FIGURE 27 – Menu de l'interface graphique

6.2 Interface du jeu

En validant le menu, on lance `initialisation()` qui initialise l'interface du jeu Mastermind. On crée :

- Un nombre nombre_couleurs de bouton permettant de séléctionner la couleur.
- un tableau de canvas (zones cliquables) qui contiendra la zone de jeu.
- Un bouton pour valider la combinaison

6.2.1 Bouton de couleurs

On utilise des boutons pour sélectionner les couleurs qu'on souhaite jouer. Lorsque les bouton sont cliqué on appelle la fonction `Selection_couleur()` qui passe la couleur cliqué en variable globale a.

6.2.2 Tableau de jeu

On utilise un tableau de canvas de taille (nombre_essais, 5) tel que, dans chaque ligne, les 4 premiers canvas contiennent les zones cliquables, et le dernier contient le résultats des évaluations.

Initialement, on fait en sorte de rendre cliquable uniquement la première ligne et par la suite uniquement la ligne correspondant au tour en cours.

Lorsqu'un canva est cliqué, on change l'image pour lui affecter la couleur sélectionnée, et on ajoute cette couleur dans la combinaison qu'on souhaite tester.

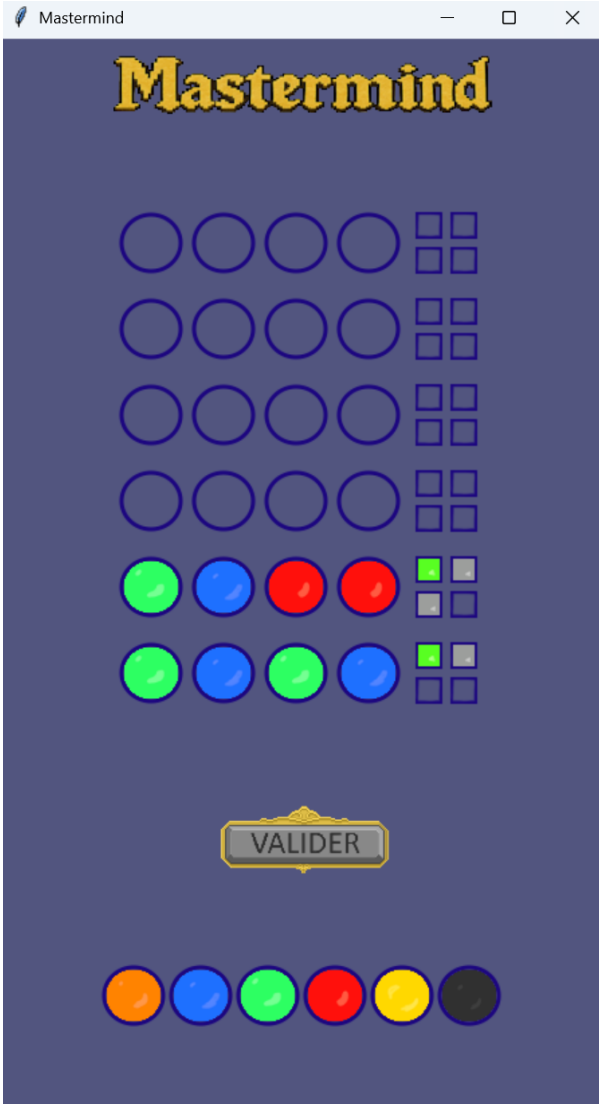

FIGURE 28 – Zone de jeu du Mastermind

6.2.3 Validation

On utilise un bouton pour valider la combinaison qu'on souhaite soumettre à l'évaluation. On fait en sorte de pouvoir appuyer sur le bouton uniquement si la proposition est complète.

Si le jeu n'est pas fini : On évalue notre combinaison avec le codemaker (1 ou 2), puis on appelle `update_évaluation` qui affiche l'évaluation de notre

combinaison, **vert** si bien placé et **gris** si mal placé. On appelle également `update_canvas`, qui change l'état de nos canvas en faisant en sorte de rentre cliquable ('bind') les canvas de la ligne supérieure à celle de la ligne actuelle et de 'unbind' les canvas des autres lignes.

Si le jeu se finit :

— Si l'utilisateur a gagné, c-à-d si son évaluation vaut (4,0) alors on ouvre la fenêtre de victoire.
— Si l'utilisateur a atteint la limite du nombre d'essais et n'a pas trouvé la solution alors on ouvre la fenêtre d'Echec.

6.3 Fenêtre de Résultat

Lorsque le jeu se finit, on lance une fenêtre de résultat (Victoire ou Échec), comme ci-dessous :

FIGURE 29 – Fenêtre de résultat.

La solution est affichée ainsi qu'un bouton « REJOUER » permettant à l'utilisateur de relancer une partie. Les fenêtres de jeu et de résultat sont alors fermées, puis la fonction `afficher_menu_mode` est appelée pour ré-ouvrir le menu.

Conclusion

Ce projet a été amusant à réaliser et nous a permis de développer nos compétences en algorithmique, en optimisation et en structuration de code à travers une approche progressive et comparative. Nous remercions nos enseignants pour leur accompagnement tout au long de ce travail.

Pour conclure, rappelons les mots inspirant de `codebreaker3` *:*

« (R, V, G, B) »